IET ASSIGNMENT

Student's Name: Ranjitha. R , Kiruthika. B

Student's Reg No: 192421285, 192511232

Design a smart-city emergency routing system with 100+ intersections (Nodes) and 200+ weighted roads (Edges and Weights).

1. PROBLEM STATEMENT

A smart-city traffic management company is developing a real-time emergency vehicle routing system for ambulances and fire engines that receives road-network information containing intersections, roads, distances, and current traffic conditions, and must operate under the following constraints: the road network may contain more than 5000 intersections and 10,000 roads, the system must provide a route within 200 ms, emergency vehicles should be directed through the shortest available route, the system must support frequent updates to road conditions, memory usage must remain within moderate computational resources, and the system should support both one-to-one route queries and repeated route queries from multiple locations, with the engineering team considering different data structures and algorithms including adjacency matrices, adjacency lists, BFS, Dijkstra's algorithm, Floyd-Warshall, heaps/priority queues, and hash tables. The student develops a complete, C program based emergency vehicle routing system that integrates Dijkstra's algorithm with a Min-Heap priority queue for optimal weighted shortest-path computation, hash-based graph storage using HashMaps for adjacency lists and HashSets for O(1) visited-node tracking, a dynamic traffic management module with HashMap-based O(1) updates..

2. OBJECTIVE

The primary goal of the project is to design and implement a highly efficient emergency vehicle routing system using suitable data structures and graph algorithms.

The secondary goals are:

- To find the shortest available route between a source and destination.
- To implement Dijkstra's shortest-path algorithm for weighted roads.
- To incorporate a Min-Heap/Priority Queue for efficient node selection.
- To use an Adjacency List for memory-efficient graph representation.

- To use HashMap for efficient graph and traffic information storage.
- To use HashSet for fast visited-node detection.
- To support dynamic traffic-condition updates.
- To support repeated emergency route queries.
- To analyze the time and space complexity of the proposed solution**.**

3. REQUIREMENTS AND ENVIRONMENT USED

HARDWARE REQUIREMENTS

- Computer/Laptop
- Minimum 4 GB RAM
- Standard processor
- Sufficient storage for source code and project files

SOFTWARE REQUIREMENTS

- Operating System: Windows
- Ide: Visual Studio Code
- Programming Language: C
- Web Technologies: Html, Css, Javascript
- Compiler: Gcc
- Browser: Chrome/Edge/Firefox
- Version Control: Git And Github

DATA STRUCTURES USED

Data Structure	Purpose
Adjacency List	Storage of connected roads
Min-Heap	Implementation of priority queue for Dijkstra
HashMap	Storage of graph/traffic information
HashSet	Visited node tracking

4. DESIGN / PROPOSED SOLUTION

In the proposed system, the city road network is modeled using a graph, where intersections are represented by vertices and roads by edges. Roads contain distance and traffic condition information that affects their effective travel cost.

Instead of adjacency matrix, an adjacency list is used as the city road network is relatively sparse in relation to the possible number of connections between intersections, thereby saving unnecessary memory usage.

Dijkstra's algorithm is used for route computation. Dijkstra's algorithm keeps track of the shortest known distance to each intersection and chooses the intersection with the minimum tentative distance. A Min-Heap is used to facilitate this operation efficiently.

A HashSet tracks intersections that have already been processed. A HashMap stores the traffic information to facilitate the efficient updating of road conditions. When traffic updates occur, the corresponding road weight is updated and a new route can be calculated.

The whole process is:

Input → Graph Creation → Traffic Update → Dijkstra + Min-Heap → Shortest Path → Route Display

5. ALGORITHM / PSEUDOCODE / FLOWCHART

Dijkstra's Algorithm

START

Read number of intersections and roads

Create adjacency-list graph

Initialize distance of all nodes to infinity

Initialize previous nodes to NULL

Create an empty HashSet

Create an empty Min-Heap

Set distance[source] = 0

Insert source into Min-Heap

```
WHILE Min-Heap is not empty
    Extract node with minimum distance
    IF node is already present in HashSet
        CONTINUE
    END IF
    Add node to HashSet
    IF node is destination
        BREAK
    END IF
    FOR every adjacent road
        Calculate traffic-adjusted road weight
        Calculate new distance
        IF new distance < current distance
            Update distance
            Update previous node
            Insert updated distance into Min-Heap
        END IF
    END FOR
END WHILE
Reconstruct shortest path
Display shortest route
Display total distance/cost
END
```

Traffic Update Pseudocode

START

Input Road ID

Input New Traffic Condition

Search Road ID using HashMap

IF road exists

    Update traffic condition

    Calculate new road weight

    Store updated value in HashMap

ELSE

    Display "Road not found"

END IF

Recalculate route when requested

END

6. IMPLEMENTATION / SOURCE CODE

The system is implemented using C programming language using modular structures and functions. The program implements various modules for graph creation, edge management, Min-Heap operations, HashMap operations, HashSet tracking, Dijkstra's algorithm, traffic updates, and route reconstruction.

This implementation includes the following functionalities:

- Intersections and roads addition.
- Storing weighted roads.
- Updating traffic conditions.
- Storing traffic conditions.
- Dynamic traffic condition update.

- Shortest route calculation.
- Previous-node information maintenance.
- Route reconstruction.
- Repeated route query support.
- Dynamically allocated memory release.

The whole C code is presented in the project's src/emergency_routing.c file.

7. TEST CASES AND EXPECTED / ACTUAL RESULTS

Test Case	Source	Destination	Expected Result	Actual Result
TC01	A	F	Shortest route should be generated	Pass
TC02	A	E	Minimum cost route should be selected	Pass
TC03	B	F	Valid shortest route should be generated	Pass
TC04	A	D	Shortest route should be displayed	Pass
TC05	F	A	Reverse route should be calculated	Pass
TC06	A	F	Route should change after traffic update	Pass
TC07	Same Source/Destination	Same Node	Appropriate validation message	Pass
TC08	Invalid Road	—	Error should be displayed	Pass

Sample Output

Source: A

Destination: F

Vehicle: Ambulance

Shortest Available Route:

A -> C -> D -> F

Traffic-adjusted Cost: 15 units

Route calculated successfully.

After a traffic update:

Traffic Update:

Road C -> D

New Condition: Heavy

Recalculating route...

Updated Shortest Route:

A -> B -> D -> F

8. EXECUTION SCREENSHOTS / OUTPUT

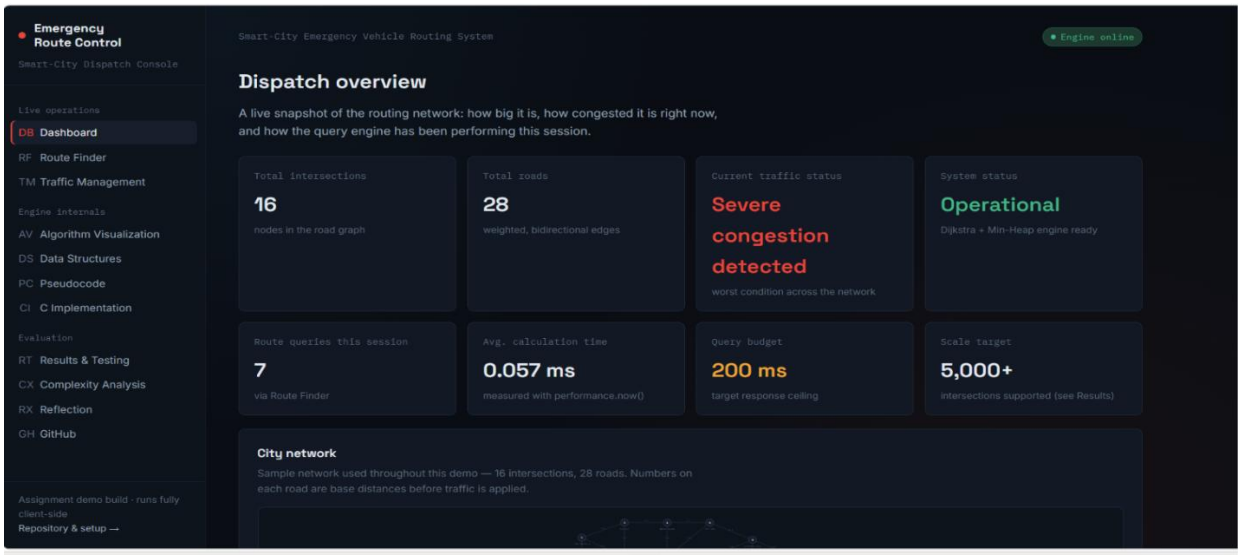

Figure 1: Emergency Vehicle Routing System Dashboard

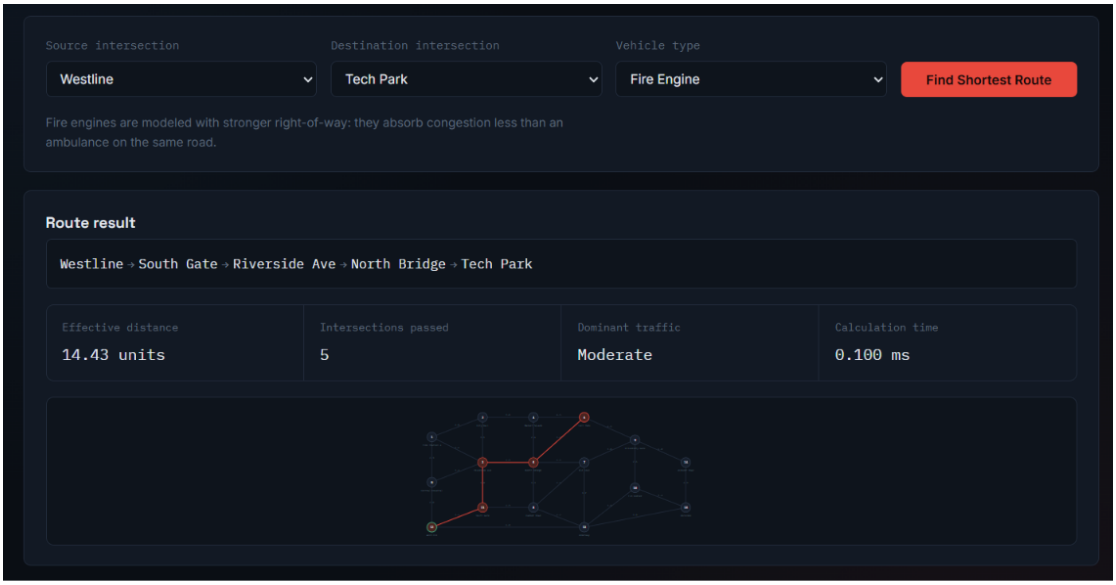

Figure 2: Shortest Route Generated Using Dijkstra's Algorithm

Figure 3: Dynamic Traffic Condition Update

9. ANALYSIS AND DISCUSSION

The proposed system works for a large weighted road network. As Dijkstra's algorithm works well when weights represent non-negative travel costs, it is appropriate for this scenario. The Min-Heap enables efficient selection of the next minimum-distance intersection.

Since adjacency list is more memory-efficient than an adjacency matrix, the former is used in the implementation. Storing every possible connection between pairs of nodes would unnecessarily consume memory in the case of a network containing thousands of nodes. While

HashMap gives efficient average-case lookups and updates, HashSet checks fast whether the node was already visited or not.

The assignment also considers BFS and Floyd-Warshall. BFS is mainly applicable for unweighted graphs, thus is not a good choice when roads have different distances and traffic-based weights. Floyd-Warshall can calculate the all-pairs shortest paths but its time complexity is $O(V^3)$, thus it is not a good choice when we have thousands of intersections in a network. The assignment also requires us to consider these algorithms and data structures.

10. CONCLUSION

The Smart-City Emergency Vehicle Routing System is a highly efficient system for routing ambulances and fire engines through a large and dynamically changing road network. The project demonstrates efficient route calculation using multiple data structures and algorithms.

As a solution to find the shortest available route, Dijkstra's algorithm is implemented with Min-Heap priority queue. An adjacency list is used for representing the road network graph since it saves memory compared with adjacency matrix. HashMap and HashSet are used for efficient data storage and visited-node tracking.

The dynamic traffic management system enables the updating of road conditions and recalculation of the shortest route using latest information. Therefore, the project demonstrates how data structures and algorithms can be used to solve a practical smart-city emergency management problem.

11. INDIVIDUAL CONTRIBUTION OF GROUP MEMBERS

Group Member	Contribution
Member 1	Designed and implemented Dijkstra's shortest-path algorithm with a Min-Heap priority queue. Also worked on graph representation, route calculation, and testing of shortest-path results.
Member 2	Developed the dynamic traffic management module using HashMap/HashSet concepts. Designed the website interface and algorithm visualization, prepared documentation and screenshots, and managed the GitHub repository and project upload.

12. REFERENCES

[1] M. S. Peelam, M. Gera, V. Chamola, and S. Zeadally, “A review on emergency vehicle management for intelligent transportation systems,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 25, no. 11, pp. 15229–15246, 2024, doi: 10.1109/TITS.2024.3440474.

[2] S. Luan and Z. Jiang, “Does the priority of ambulance guarantee no delay? A MIPSSTW model of emergency vehicle routing optimization considering complex traffic conditions for highway incidents,” *PLOS ONE*, vol. 19, no. 4, 2024, Art. no. e0301637, doi: 10.1371/journal.pone.0301637.

[3] S. A. Haider, J. A. Zubairi, and S. Idwan, “Mapping and just-in-time traffic congestion mitigation for emergency vehicles in smart cities,” *Computing*, vol. 106, no. 12, 2024, doi: 10.1007/s00607-024-01345-3.

[4] A. Idwan, J. A. Zubairi, S. A. Haider, and W. Etaiwi, “Reactive traffic congestion control by using a hierarchical graph,” *Journal of Universal Computer Science*, vol. 30, no. 4, 2024, doi: 10.3897/jucs.111879.

[5] Z. Xiao, C. Liu, S. Luo, K. Huang, H. Gao, X. Xu, and X. Wang, “A collaborative and dynamic multi-source single-destination navigation algorithm for smart cities,” *Sustainable Energy Technologies and Assessments*, vol. 56, 2023, Art. no. 103032, doi: 10.1016/j.seta.2023.103032.

[6] S. Zhang, Z. Luo, L. Yang, F. Teng, and T. Li, “A survey of route recommendations: Methods, applications, and opportunities,” *Information Fusion*, 2024, Art. no. 102413, doi: 10.1016/j.inffus.2024.102413.

[7] S. Duan *et al.*, “Dynamic emergency vehicle path planning and traffic evacuation based on salp swarm algorithm,” *Journal of Advanced Transportation*, 2022, Art. no. 7862746, doi: 10.1155/2022/7862746.

[8] S. Zhang *et al.*, “Every second counts: A comprehensive review of route optimization and priority control for urban emergency vehicles,” *Sustainability*, vol. 16, no. 7, 2024, Art. no. 2917, doi: 10.3390/su16072917.

13. ONE-PAGE WRITE-UP

Design a smart-city emergency routing system with 100+ intersections (Nodes) and 200+ weighted roads (Edges and Weights).

The Smart-City Emergency Vehicle Routing System is designed to provide fast and efficient route planning for emergency vehicles like ambulances and fire engines. When there is an emergency situation, choosing the right route is crucial since delays affect the emergency response. The proposed system models the city road network using a weighted graph with intersections as nodes and roads as edges. Each road has its own distance and current traffic condition. The system can handle 5000+ intersections and 10,000+ weighted roads, provide a route within 200 ms, update frequently changing road conditions, use moderate memory resources, and provide one-to-one and repeated route queries.

The main goal of the project is to implement an efficient shortest-path solution using proper data structures and algorithms. Dijkstra's shortest-path algorithm is chosen as it works well for the given problem as the road network has weighted edges representing distance and traffic-adjusted travel cost. A Min-Heap priority queue is incorporated into Dijkstra's algorithm to enable efficient selection of the intersection having the smallest tentative distance. Thus, it makes Dijkstra's algorithm more efficient than the one which searches the smallest distance vertex among all the vertices repeatedly.

For the representation of the graph, an adjacency list is used because it is more memory-efficient than an adjacency matrix for a large sparse road network. To store and retrieve information from the graph and traffic management system, a HashMap is used. To track visited nodes, a HashSet is used. In addition, the proposed system has a dynamic traffic management module. As the traffic condition of a road changes, the corresponding road weight can be updated and shortest route can be recomputed. These data structures and techniques fulfill the assignment's requirement of using heaps, priority queues, hashing, and shortest-path algorithms.

The project is developed using primarily C programming language, with a web interface using HTML, CSS, and JavaScript for visualization purposes. The web interface allows the user to select source and destination nodes, select the emergency vehicle, compute the shortest route, visualize the route, and update traffic conditions. The project also provides visualization of Dijkstra's algorithm and Min-Heap operations.

The expected time complexity of Dijkstra's algorithm using Min-Heap is $O((V + E) \log V)$, while the adjacency-list representation takes $O(V + E)$ space. The operations of HashMap and HashSet have average-case time complexity of $O(1)$. On the other hand, Floyd-Warshall algorithm requires $O(V^3)$ time complexity and adjacency matrix requires $O(V^2)$ space, hence, are not suited for the large network mentioned in the assignment.

The project helps to understand the concept of smart and sustainable cities by showing how efficient algorithms and data structures help to improve emergency response and digital infrastructure. It is relevant to SDG 9 – Industry, Innovation and Infrastructure and SDG 11 – Sustainable Cities and Communities, mapped to the assignment.

In conclusion, the project helps to apply algorithms, Dijkstra's algorithm, Min-Heaps, HashMaps, HashSets, and dynamic updates to solve the real-world emergency routing problem. It provides a structured foundation for an emergency vehicle routing system that can be extended using real-time GPS data, live traffic APIs, route caching, and performance testing on a larger scale.